

# How SigLIP2 / OpenCLIP Fine-Tuning Works

Theory behind 01\_experiment\_fine\_tuning.bash - costume/franchise image-caption alignment

## 1. The architecture: two encoders, one shared space

---

SigLIP2 (like CLIP before it) is a dual-encoder model: it has an image encoder (a Vision Transformer, ViT-B/16) and a text encoder (a Transformer), and no shared layers between them. Each encoder maps its input into a vector in the same D-dimensional embedding space:

```
x_i = image_encoder( image_i )    -> vector in R^D
y_i = text_encoder( caption_i )    -> vector in R^D
```

Both vectors are then L2-normalized so only their direction matters, not their magnitude. Training does not teach the model to 'read' captions as labels in a classifier sense - there is no fixed softmax output layer over classes. Instead, it teaches the two encoders to place matching image/caption pairs close together in this shared space, and non-matching pairs far apart. This is why the same model that was fine-tuned on your costume captions can still be queried with captions never seen during training - it generalizes via geometry, not memorized class indices.

## 2. Where the pairs come from: train\_metadata.csv

---

Each row of train\_metadata.csv is one (filepath, caption) pair, e.g. an image of a costume plus its label such as "a cosplay costume of Frieren from Frieren". --csv-img-key filepath and --csv-caption-key caption in the bash script simply tell open\_clip\_train which columns to read.

During training, the dataloader forms mini-batches of N such pairs (N = --batch-size 32, per GPU). Within a batch:

- the image and caption from the SAME row form a positive pair (they should align)
- the image from row i and the caption from any OTHER row j in the batch form a negative pair (they should NOT align)

So alignment is defined relationally, within each batch, not via a fixed lookup table of classes. If two different rows in the same batch happen to share the exact same caption (e.g. two images both captioned "a cosplay costume of Frieren from Frieren"), that image is effectively also a soft negative for the other row's positive text, and vice versa - the model still learns something useful from it because the loss only cares about matching image/caption INDEX, not deduplicated captions.

## 3. The similarity score

---

For every image i and every caption j in the batch, the model computes a similarity score - just the cosine similarity (dot product of the normalized vectors) scaled by a learned temperature t and, for SigLIP specifically, shifted by a learned bias b:

```
z_ij = t * (x_i . y_j) + b
```

t and b are themselves trainable scalars (not hyperparameters you set in the bash script) - the model learns how confidently to separate matches from non-matches as training progresses.

## 4. The loss: what error is actually minimized

---

This is the key theoretical difference between original CLIP and SigLIP2. Original CLIP uses a softmax contrastive loss (InfoNCE): for each image, it runs a softmax over ALL captions in the batch and asks 'did the model put most of the probability mass on the correct one', and the same in the other direction for each caption over all images. That requires normalizing across the whole batch on both axes, which is part of why CLIP needed very large batch sizes to work well.

SigLIP replaces this with a much simpler, batch-size-independent formulation: it treats every single (image, caption) pair in the batch as an independent binary classification problem - 'do these two match, yes or no' - solved with a sigmoid, not a softmax:

```
label_ij = +1    if caption j is the true caption of image i
label_ij = -1    otherwise (negative pair)

L = -(1/N) * sum_i sum_j log( sigmoid( label_ij * z_ij ) )
```

*N = batch size (32 in this script). The double sum runs over every image-caption combination in the batch: N positive pairs (the diagonal) and N\*(N-1) negative pairs (everything off the diagonal).*

Intuitively: for a true (image, caption) pair,  $\text{sigmoid}(z_{ij})$  should be pushed towards 1 (log-loss minimized when  $z_{ij}$  is large and positive). For a mismatched pair,  $\text{sigmoid}(z_{ij})$  should be pushed towards 0 (loss minimized when  $z_{ij}$  is large and negative). Gradient descent on this loss nudges the image and text encoder weights so that matching embeddings move closer together (higher cosine similarity) and non-matching embeddings move further apart, batch after batch, epoch after epoch.

Practically this means: there is no single 'ground truth class error rate' being minimized during training itself (that only shows up later, at eval time, in the top-1/top-5 accuracy computed by `00_evaluate_baseline.py`). What is directly minimized during training is this sigmoid pairwise loss over image/caption similarity scores.

## 5. Fine-tuning vs. training from scratch

---

The bash script does not build a model from random weights - `--model hf-hub:timmm/ViT-B-16-SigLIP2-256` loads a checkpoint already pretrained on a huge general image-text dataset. Both encoders keep their pretrained weights as the starting point and continue training (no `--lock-image` or `--lock-text` flag is passed, so both towers stay trainable - nothing is frozen).

Two settings in the script exist specifically to protect that pretrained knowledge instead of overwriting it wholesale:

- `--lr 1e-6` - a learning rate roughly 100-1000x smaller than a from-scratch run would use, so weight updates are small nudges rather than large rewrites
- `--warmup 50` - the LR ramps up gradually over the first 50 steps rather than jumping straight to 1e-6, avoiding a destructive gradient spike at the very start when the optimizer state is still empty

The effect: the general visual/semantic understanding SigLIP2 already has (shapes, colors, textures, generic 'a photo of a person') is preserved, while the geometry of the embedding space gets specialized so that your specific costume/franchise captions pull the corresponding images noticeably closer than they were pretrained to be. 5 epochs (`--epochs 5`) over `train_metadata.csv` is enough passes to shift the space without the low LR requiring many more.

## 6. Putting it together: one training step

---

- Sample a batch of 32 (image, caption) rows from `train_metadata.csv`
- Encode all 32 images -> 32 normalized vectors  $x_i$ ; encode all 32 captions -> 32 normalized vectors

$y_j$

- Compute the 32x32 similarity matrix  $z_{ij} = t * (x_i \cdot y_j) + b$
- Compare against the 32x32 label matrix (+1 on the diagonal, -1 elsewhere)
- Compute the sigmoid loss  $L$  and backpropagate through both encoders (and  $t$ ,  $b$ )
- Optimizer (AdamW, --wd 0.1) updates every trainable weight by a small step scaled by the warmed-up  $lr=1e-6$
- Repeat for every batch, every epoch; save a checkpoint every epoch (--save-frequency 1) and log the loss to Weights and Biases (--report-to wandb)

Reference: `openCLIP/finetuning/01_experiment_fine_tuning.bash` (hyperparameters),  
`openCLIP/finetuning/00_evaluate_baseline.py` and `00_test.py` (how the resulting embeddings are later used for zero-shot classification via cosine similarity).